

C02 — Product Thinking and Decision Foundations

1. What interviewers test

Every product interview, whatever its label, tests one thing underneath: can you move from an open-ended situation to a decision, and can you show what that decision costs? The label changes (product sense, execution, strategy, behavioral), but the scorer is listening for the same few signals.

Where it shows up. Inferred from the round structure candidates commonly report: explicit tradeoff questions ("ship now or polish?"), "how would you decide whether to build X?", why-probes into a past decision ("why that option and not the other?"), and silently in every design or execution answer, where the interviewer notes whether you chose and justified or merely listed. On the job, the same skill appears in your first month: feature requests from sales, a bug the CEO noticed, two engineers who disagree, a deadline that moved.

What is scored. The attributes companies state publicly point the same way. Google describes hiring on general cognitive ability, role-related knowledge, leadership and "Googleness" [Public \[V: summaries citing Google's how-we-hire page; Google re:Work structured-interviewing guide\]](#), and structured reasoning to a decision is how the first two are observed in a product round Inferred. Amazon publishes its Leadership Principles, including Are Right, A Lot, Bias for Action and Dive Deep [Public \[V: Amazon's Leadership Principles page, amazon.jobs\]](#). A scorer's written note usually reads like "chose an approach and defended it" or "listed options, never committed" Inferred.

Five observable signals, which the rest of the chapter trains:

1. You state the decision and the goal before you generate options.
2. You separate the problem from the proposed solution.
3. You name what each option costs, in a sentence that begins "we get X by giving up Y."
4. You say how reversible the decision is and size your evidence accordingly.
5. You say what evidence would change your mind.

The failure this prevents. Four patterns lose candidates and, later, new PMs: solution-first answers ("add a chatbot"), indecision dressed as analysis ("it depends on many factors"), precision theater (a score of 7.3 with no stated basis), and options with no cost attached, which signals that the cost was never looked for.

2. The concept from first principles

C02.1 Product thinking defined

Product thinking is reasoning from a user problem to a solution to a measurable outcome, rather than starting from a solution. `[[product-thinking]]` has four moves, always in this order: who has the problem and in what situation; why it matters, stated as an outcome you could measure; what options exist, including doing nothing; and what evidence would tell you which option works.

Why it works: solutions are cheap to generate and expensive to build, and the problem is the constraint that prunes the option space. A team that starts from "add a chatbot" has already spent its most valuable resource, the freedom to choose a cheaper fix. A team that starts from "customers who cannot find the return policy contact support three times more often" can consider a link, a search tweak, a chatbot, or a policy change, and compare them.

Two companion ideas are owned by later chapters and linked here rather than redefined. A `[[problem-statement]]` is the written form of the first two moves (C06). `[[jobs-to-be-done]]` is a lens for the first move, describing the progress a user is trying to make (C06).

When not to use it. Product thinking has a cost: the time spent framing. Skip the full chain when the decision is cheap and reversible and the need is obvious (fixing a typo, correcting a misleading label), when a legal or contractual obligation fixes the requirement, or when you are copying a pattern with proven value in your context, such as a standard password-reset flow.

Counterexample. A team spends three weeks framing a one-line copy change on a checkout button. The decision is reversible in minutes and measurable the same day, so the correct process is to ship and watch the metric. Product thinking applied to a cheap, reversible choice is waste, and an interviewer will notice if you apply the same heavy structure to every prompt regardless of stakes.

Alternatives. Solution-first work is legitimate when you hold strong prior evidence from the same context, and in discovery sprints where building is the research method. The risk is that you will not notice when the evidence stops applying.

Hidden risks and second-order effects. The chain can be turned into a justification device: you pick the solution first, then write the problem so it fits. A tell is that the problem statement contains the solution's nouns ("users need a pause button"). A second-order effect of good framing is that it makes requests visible as symptoms, which changes how you talk to requesters; requesters hear a refusal of the feature as a refusal of the need unless you restate the need.

C02.2 First-principles reasoning and when analogy is better

First-principles reasoning is breaking a problem into its basic facts and rebuilding the answer from them instead of copying an existing solution. [[first-principles-reasoning]] has a procedure, not a mood:

1. State the goal in terms of an outcome.
2. List the facts that are fixed: physics, law, measured user behavior, hard cost floors.
3. List the assumptions that are only conventions: "onboarding means a training session", "pricing is per seat", "we must support every device".
4. Rebuild a solution from the fixed facts, and challenge each convention you keep.

Analogy, copying what works elsewhere, is faster and usually right in well-trodden problems. Use analogy when the domain is mature and your context resembles the source. Use first principles when a constraint has changed (a technology made something cheap that used to be expensive), when two analogies give opposite answers, or when you suspect an inherited convention is the real bottleneck.

Hidden risks. You can mistake a convention for a fact, or a fact for a convention. A team that re-derives a payment flow from scratch and treats card-network rules as negotiable will fail late and expensively. You can also spend a week re-deriving what is already known. First principles is a tool for the part of the problem where analogies fail, not a replacement for knowing the field.

Counterexample. A candidate asked to design an identity-verification step "from first principles" ignores that regulators in many markets prescribe what must be collected. The fixed-facts list was incomplete, so the elegant redesign was not allowed. Before you drop a convention, ask whether a rule you have not met enforces it.

C02.3 Tradeoffs: the common axes and how to make them visible

A **tradeoff** is a choice in which gaining on one dimension costs something on another, stated with what is given up. [[tradeoff]] is the heart of PM judgment because resources are limited and users, the business and engineering rarely want the same thing.

The axes that recur in product work:

Axis	Typical form	Note
Speed versus quality	Ship in two weeks versus six	Often hides the real variable: which defects, for whom
Scope versus time	Fewer features, earlier	Cut scope before cutting quality when you can
Short term versus long term	Growth tactic now, trust cost later	Most [[goodharts-law]] damage lives here (C05)
Breadth versus depth	Support many cases versus one case well	Relevant to every first release

Axis	Typical form	Note
Simplicity versus power	Fewer controls versus configurability	Different users sit at different points
Growth versus trust	Aggressive prompts versus user goodwill	Trust is slow to rebuild
Privacy versus personalization	Less data versus better relevance	Often a design problem, not a pure tradeoff
Consistency versus flexibility	One way versus per-team variation	Platform and design-system decisions
Build versus buy versus partner	Control versus speed and cost	Owned in depth by C18.7 (the chooser) and C21.10 (the technical side)
Cost versus performance	Cheaper infrastructure versus latency	Frequent in AI features (C24)

Making a tradeoff visible. Put the sacrifice in the sentence. "We recommend shipping the simplified checkout, which gives up saved-address editing for the first release." Three habits help: write what you give up before what you gain; name who bears the cost; and say whether the cost is reversible. A choice with no stated cost is unfinished.

The bad-decision drill. Before committing, ask: "What would have to be true for this to be a bad decision?" Each answer is either a risk to monitor, a condition that changes your recommendation, or a fact you can check now.

False tradeoffs. Some "speed versus quality" pairs dissolve when you find the actual cause. If the slow part is a review queue, fixing the queue buys speed without cutting quality. Test every either-or by asking what specifically makes the fast option worse, and whether that can be addressed separately.

C02.4 Decision tools: reversibility, cost of delay, confidence, why now

Reversibility. A **reversible decision** is one that can be undone cheaply, so it can be made quickly on less evidence. `[[reversible-decision]]` is the single most useful classifier because it sets how much evidence you should demand. Changing button text, running a one-week test, or enabling a `[[feature-flag]]` for 5% of users is reversible. Retiring a public API that customers integrated against, changing a price that customers have already seen, or shipping a hardware design that goes to manufacturing is hard or impossible to undo. Reversibility is a matter of degree and of audience: a change may be technically reversible while the reputational cost of reversing it is high (see CS-C02-02).

Cost of delay. The **cost of delay** is the value lost for each unit of time a decision or release is postponed. `[[cost-of-delay]]` puts a price on waiting. A fix that recovers revenue worth 1% of weekly sales loses that amount every week it sits in the backlog. Use it to resist the common pattern of making cheap decisions slowly. It also exposes decisions that feel urgent but cost little to delay.

Confidence levels. State how sure you are and what would raise or lower it. A label (low, medium, high) with a reason beats a false number: "medium confidence, because we have usage data but no interviews with the affected segment." If you give a percentage, say it is a judgment. Tie confidence to the cost of being wrong: low confidence is acceptable on a reversible decision and a reason to gather evidence or stage the decision when it is not.

The why-now test. Ask what has changed that makes this the right time. Answers include a new capability, a competitor move, a metric trend, a dependency that cleared, or a deadline. A decision with no why-now is usually a preference and can wait.

When not to use these tools. Do not run a formal classification for decisions you can reverse in minutes; the time you spend classifying exceeds the cost of being wrong. Do not treat confidence labels as precision; they are communication aids. Do not use cost of delay as a reason to ship something unready; delay cost is one input beside the cost of error.

C02.6 Taste and judgment

Taste here means a trained ability to recognize quality and fit with a user's need quickly, before data arrives. **Judgment** means choosing well when evidence is incomplete. Neither is innate. Inferred. Both develop through the same loop: exposure to many examples with explicit analysis, predictions made before outcomes are known, and honest comparison of the two.

Four drills for building both:

1. **The ten-minute teardown.** Pick one screen you use. Write three things it does well and three it does badly, each with the user need it serves or fails. Reasons, not preferences: "the primary button is below the fold on small phones, so the main action needs a scroll" rather than "it feels cluttered."
2. **The prediction log.** Before an experiment readout or launch, write your prediction and your confidence. After the result, score it. Over a quarter you learn where your intuition is reliable and where it is not.
3. **The decision journal.** For each non-trivial decision, record the context, the options, your choice, your confidence and what would change your mind. Review after the outcome, separating decision quality from outcome luck.
4. **Remove one thing.** Take a product you know and say what you would remove and why. Removal forces a view of what matters.

What good looks like. You can give a reason for a judgment, you can say when you are likely to be wrong, and you change your mind quickly when evidence arrives.

Counterexample. Strong taste misleads when you are not the user. A designer who loves a dense, keyboard-driven interface will misjudge a product for first-time smartphone users. Taste is a hypothesis about someone else's experience until you have checked it (the [[say-do-gap]], C06).

C02.7 Templates: decision memo, problem definition, pre-mortem

Three short templates put the habits of this chapter on the page. The **decision memo** (FW-C02-06) records a decision so that it can be reviewed later: the question, the options, the choice, the reasoning and what would change it. The **pre-mortem** (FW-C02-05) lists how a plan could fail before it starts. The third is a four-line **problem definition**, which stops a team from solving a problem nobody stated:

1. *Who has the problem*, and roughly how many people.
2. *What happens today*, described as something observable, with no solution in it.
3. *The cost*, as a number or a named consequence, and the evidence for it.
4. *What we do not know yet*, and the cheapest way to find out.

For example: "New users who sign up on a phone abandon the app in the first session 55% of the time (invented); recordings show most stop at the permission prompts. We do not yet know whether the prompts or the empty home screen matter more; a five-user test would tell us." C06 extends this form into a full problem statement with evidence from discovery ([[problem-statement]]). Use the templates when a decision will be revisited or shared. For a quick, reversible choice, one sentence does the job.

3. Frameworks

Six frameworks, set out below. The first three are working tools for any prompt; the last three structure decisions and documents.

FW-C02-01 · The product thinking chain · Original

- **Inputs:** a request, idea or metric target, plus whatever you can observe about the people behind it.
- **Outputs:** a [[problem-statement]] (C06), a measurable outcome, an option set that includes doing nothing, and an evidence plan.
- **Assumptions:** the request has a need behind it that you can identify; you can reach or observe the users involved.
- **Tradeoffs:** time spent framing versus building; risk of over-framing cheap decisions.
- **Failure modes:** writing the problem to fit a chosen solution; endless why-chains with no decision; treating the requester as the user; skipping the "do nothing" option.
- **Worked example:** a meal-kit app is asked for a "pause subscription" button. The chain converts it to "subscribers with variable schedules cancel because skipping a week needs a support call," sets 60-day retention of that segment as the outcome, and generates four options. See Worked example 1.

The chain in steps: (1) restate the request as a need ("what is this person trying to do?"); (2) identify the [[user-segment]] and the situation; (3) write the outcome as a [[metric]] with a direction and a window; (4) list options of different kinds, including a smaller one and doing nothing; (5) state the evidence you have, the evidence you lack, and the cheapest way to get it.

Problem-definition template (an input to the chain, expanded in C06):

Field	Prompt
Who	Which [[user-segment]], in what situation
What happens today	The observable behavior or friction
Why it matters	The outcome affected, as a [[metric]]
Evidence	What you know, and how well
Not the problem	The solution or feature the request named

FW-C02-02 · First-principles decomposition · Original

- **Inputs:** a goal stated as an outcome; the current solution and its conventions.
- **Outputs:** two lists (fixed facts and inherited conventions) and a rebuilt design that challenges each convention.
- **Assumptions:** you can distinguish what is physically, legally or empirically fixed from what is merely customary; you will check any "fixed" item you are unsure of.
- **Tradeoffs:** deeper insight versus time; risk of reinventing known solutions.
- **Failure modes:** treating a regulation as a convention; treating a convention as a law; confusing a rebuilt design with a better one without evidence.
- **Worked example:** an expense-management product's three-week onboarding is decomposed into fixed facts (employee data, a policy, a verified bank connection) and conventions (live training before use, full policy configuration up front). The rebuild defers training until after the first reimbursement run. See Worked example 2.

FW-C02-03 · The tradeoff table · Original

- **Inputs:** two to four options; the goal; the constraints (time, people, risk).
- **Outputs:** a table recording, for each option, what it optimizes, what it costs and who bears the cost, how reversible it is, and what evidence it needs; plus a recommendation with the cost stated.
- **Assumptions:** the options are genuinely different in kind; the criteria come from the goal, not from habit.
- **Tradeoffs:** clarity versus the time needed to fill it in honestly; a table can imply precision that is not there.
- **Failure modes:** an option column with no costs; criteria added after the favorite is chosen; weighting that hides the real sacrifice.
- **Worked example:** for a payments app, three candidate launches are compared on speed, risk to data integrity and reversibility; the table shows that the fastest option's cost falls on customers' money, which rules it out. See Worked example 3.

Option	Optimizes	Costs (and who bears it)	Reversibility	Evidence needed
A				

Option	Optimizes	Costs (and who bears it)	Reversibility	Evidence needed
B				
Do nothing				

Recommendation line: "I recommend B, which gives us ___ by giving up ___. This is [reversible / hard to reverse] because ___. I would change my mind if ___."

FW-C02-04 · The decision classifier · Established

- **Origin:** one-way and two-way door decisions, as described in Amazon's shareholder letters in the mid-2010s [V: Bezos 2015 shareholder letter (published April 2016), repeated in the 2016 letter]; cost of delay as an economic input to product decisions, from Donald Reinertsen, *The Principles of Product Development Flow* (2009). Combining them into one three-question classifier is this book's arrangement.
- **Inputs:** a decision; estimates of how reversible it is, what an error costs, and what waiting costs.
- **Outputs:** one of three modes: decide now with light evidence; decide with evidence and a plan to verify; stage the decision (pilot, flag, notice period) so it becomes reversible.
- **Assumptions:** you can estimate the three quantities roughly; reversibility is judged for all affected audiences, not only for engineering.
- **Tradeoffs:** a quick classification costs minutes, and misclassifying a one-way decision as reversible is costly.
- **Failure modes:** classifying by how hard the change is to code rather than how hard it is to undo for users; treating cost of delay as the only input; never revisiting the class when facts change.
- **Worked example:** three decisions in one week at a payments app (a message wording, a refund-window change, a legacy API shutdown) land in three different modes. See Worked example 3.

Reversible?	Error costly?	Waiting costly?	Mode
Yes	Low	Any	Decide now; test if cheap
Yes	High	Yes	Decide now, [[staged-rollout]] with a rollback trigger
No	High	Low	Decide slowly with evidence; stage or give notice
No	High	High	Reduce irreversibility first (pilot, flag), then decide

FW-C02-05 · The pre-mortem · Established

- **Origin:** Gary Klein, "Performing a Project Premortem," *Harvard Business Review*, September 2007.
- **Inputs:** a plan or launch about to be committed; the people who will execute it.
- **Outputs:** a ranked list of reasons the plan could fail, with mitigations, monitoring signals and, where needed, changes to scope or launch criteria.

- **Assumptions:** participants will state concerns when asked to imagine failure; someone owns acting on the list.
- **Tradeoffs:** thirty to sixty minutes of team time versus catching a failure before it is expensive; it can surface fear that stalls a team if not closed with decisions.
- **Failure modes:** treating it as a risk-register ritual; letting the most senior voice set the list; producing risks with no owner or action.
- **Worked example:** a family location-sharing feature is imagined to have failed six months after launch; the failure list surfaces misuse, teen workarounds and privacy backlash, and changes the launch criteria. See Worked example 4.

Steps: (1) tell the team the launch has failed, spectacularly, six months from now; (2) each person writes reasons independently; (3) collect and cluster; (4) rank by likelihood and impact; (5) for each top item decide a mitigation, a signal and an owner; (6) change the plan.

FW-C02-06 · The decision memo · Original

- **Inputs:** a decision that needs agreement or a record; the evidence gathered.
- **Outputs:** a one- to two-page document: decision, context, options, criteria, recommendation, tradeoffs accepted, reversibility, what would change the recommendation, owner and date.
- **Assumptions:** readers will read a short document and the decision owner will act on it.
- **Tradeoffs:** writing time versus repeated meetings; a memo can freeze a decision that should stay open.
- **Failure modes:** burying the recommendation; options written as straw men; no statement of what would change the decision.
- **Worked example:** the refund-window decision from Worked example 3 written as a memo, ending with "revisit if refund-fraud cost exceeds the threshold for two consecutive months (invented)."

Template, in the order a reader needs it:

1. **Decision needed and by when.**
2. **Recommendation** (one sentence, with the main cost).
3. **Context** (what changed; why now).
4. **Options considered** (including doing nothing), each with what it optimizes and costs.
5. **Criteria** and how each option scores.
6. **Reversibility and the plan if we are wrong.**
7. **What would change this recommendation.**
8. **Owner, next step, review date.**

4. Answer architecture

Two question types use this chapter directly. The same structure carries over, with additions, to the design and execution rounds covered in C07, C15 and C17.

Type A: the tradeoff or "should we build it" question

Example prompts: "A feature could ship a week early with minor bugs. Do you ship?" "Sales wants a custom report for one large customer. Do you build it?"

Structure

1. **Restate** the decision and who decides.
2. **Clarify** the goal, the constraints, the audience affected, the timeline, and how reversible the decision is.
3. **Frame the options**, including a smaller version and doing nothing.
4. **Set criteria** drawn from the goal (and say which criterion matters most).
5. **Compare** with tradeoffs named in the sacrifice sentence.
6. **Classify** reversibility and cost of delay; choose the mode (FW-C02-04).
7. **Recommend**, state the cost, and say what would change your mind.
8. **Close** with the first step and how you would know early if you were wrong.

Time budget for a ten-minute segment inside a 45-minute round

Step	Minutes	What to produce
Restate and clarify	1.5	Goal, constraints, reversibility, who decides
Frame options	1.5	Two to four options of different kinds
Criteria and compare	3	Named tradeoffs; the deciding criterion
Classify and recommend	2	Mode, recommendation, stated cost
Close	1	What would change your mind; first step; early signal
Slack for interviewer questions	1	

What to clarify first. The goal behind the decision, the people affected, what has already been committed, how soon the answer is needed, and what happens if the decision is wrong.

Where to signpost. After clarifying: "I will compare three options against the goal, then recommend." Before the recommendation: "My deciding criterion is X."

Where to state tradeoffs. In the comparison and again in the recommendation line. The interviewer should hear the sacrifice twice.

How to close. One sentence: "I recommend B, which gets us ___ by giving up ___; it is reversible through ___; I would switch if ___."

Type B: "Walk me through a decision you made" and why-probes

This is the behavioral form of the same skill and is covered in full in C31. The decision-story structure is: context and goal, options you considered, the criteria, the decision and its cost, the result, and what you would do differently. When an interviewer probes ("why not the other option?"), answer with the criterion that decided it and the evidence you had at the time, not with the outcome.

Type C: the why-probe in any round

When the interviewer says "why?", resist defending; respond with the reasoning in three parts: the goal you were serving, the option you rejected and its cost, and the evidence that would have changed your answer. This shows that your preference follows from reasoning rather than the other way round.

5. Worked examples

All four examples are **Illustrative**: the products are fictional and every number is invented.

Worked example 1 — Consumer: the "pause subscription" request (Illustrative)

Setting. A meal-kit app receives about 140 support tickets a month (invented) asking for a way to pause. A sales lead says: "Add a pause button."

Step 1: restate the request as a need. The people asking want to stop receiving boxes for a while without losing their account and preferences. That is the need. "Pause button" is one way to meet it.

Step 2: find the segment and the situation. A sample of 60 tickets (invented) gives travel 24, budget 15, "too many boxes this week" 12, other 9. The concentration is in subscribers whose availability varies week to week. A quarter of last quarter's cancellations (invented) came from subscribers who had asked support about skipping a week in the prior 14 days.

Step 3: write the outcome. Raise 60-day retention for subscribers who ask about skipping. That is a [[metric]] with a segment, a direction and a window.

Step 4: generate options of different kinds.

Option	Optimizes	Costs	Reversibility
A. Pause up to three months	Widest coverage of absences	Pausers may drift into silent churn; billing and inventory complexity	Hard once subscribers rely on it
B. Self-serve "skip a week" with a cutoff reminder	The stated cause (uneven schedules)	Does not help long absences	High; behind a [[feature-flag]]
C. Flexible delivery day	Fewer skips needed	Logistics cost	Medium

Option	Optimizes	Costs	Reversibility
D. Do nothing; give support a skip macro	No build cost	Support load stays; no evidence of demand	High

Step 5: evidence plan and decision. Ship B to 20% of the segment and compare 60-day retention with the rest. Add a [[counter-metric]]: revenue per subscriber, so that skipping is not just a quieter way to lose revenue. Cost of delay is modest (140 tickets at an assumed \$6 handling cost is \$840 a month, invented, plus cancellations), so a short test is justified. Revisit A if B adoption is high and exit surveys still cite long absences.

What this shows. The request named a solution. The chain turned it into a problem, produced a smaller reversible option, and defined how to learn. Note the check on the survey evidence: ticket reasons are what people say, so the plan also looks at what they do (the [[say-do-gap]], C06).

Worked example 2 — B2B: rebuilding onboarding from first principles (Illustrative)

Setting. An expense-management product takes a median of 21 days (invented) for a new customer to complete its first reimbursement run. The team's plan is a better training deck.

Decompose, then label each step as fixed or convention.

Step	Days (invented)	Fixed fact or convention?
Collect employee list and approvers by spreadsheet	4	Data is needed (fixed); the spreadsheet is a convention
Configure the full expense policy	6	At least one rule is needed (fixed); configuring everything up front is a convention
Connect and verify the bank account for payouts	5	Bank verification is required [as of 2026-10; varies by market] (treated as fixed)
Live admin training sessions	4	Convention
Custom report setup	2	Convention

Rebuild. The goal is "a finance admin completes a first reimbursement run." From the fixed facts alone: sync employees from the company's directory instead of a spreadsheet (one day); apply a default policy template that the admin can edit later (same day); start bank verification on day one in parallel; teach through in-product guidance after the first run; defer reports. The critical path becomes the five-day bank verification plus one day, so about six days (invented).

Check the rebuild. What could break? Default policies may misfit some companies, so show the defaults for approval before the first run. Directory sync may not exist for small customers, so keep a spreadsheet fallback.

What this shows. A training deck would have improved a convention. Decomposition showed that most of the 21 days was convention, and that the one real floor, bank verification, should start first. It also shows where first principles stops: the five-day step did not go away.

Worked example 3 — Fintech: classifying three decisions in one week (Illustrative)

Setting. A payments app's PM faces three decisions this week.

Decision	Reversible?	Error costly?	Waiting costly?	Mode (FW-C02-04)
a. Rework the failed-payment message	Yes, in minutes	Low	Low	Decide today; test if cheap
b. Cut the refund window from 7 days to 3	Partly: customers will have seen the policy	High: trust and possible rule constraints [varies by market]	Moderate: refund losses of 12k a month (invented)	Stage the decision
c. Retire the legacy API used by 40 merchants (invented)	No: merchants must migrate	High: broken payments	Moderate: upkeep ties up two engineers (invented)	Decide with evidence; give notice

Decision b in detail, using the tradeoff table.

Option	Optimizes	Costs and who bears them	Reversibility
Keep 7 days	Customer goodwill	Refund losses continue (business)	Full
Cut to 3 days for everyone	Loss reduction	Trust cost to existing customers	Hard once announced
Cut to 3 days for new customers only	Loss reduction with less trust risk	Two policies to explain and operate	Medium
Tier by risk score	Targets the loss	Fairness and explanation burden; needs engineering	Medium

Recommendation line. "I recommend 3 days for new customers only, which cuts refund losses over time by giving up some policy simplicity. It is reversible through a review date, and I would extend it to existing customers if refund losses stay above the threshold for two consecutive months (invented) and complaint volume stays flat."

Decision c. Publish a deprecation date with a long notice period, a migration guide, and a named contact, and measure how many merchants have migrated at each checkpoint. The irreversibility sits with merchants, so the process, not the speed, carries the weight.

What this shows. Classification took minutes and produced three different speeds. Decision (a) would have been a mistake to over-process, and (c) a mistake to rush.

Worked example 4 — Consumer safety: a pre-mortem for family location sharing (Illustrative)

Setting. A family app plans to launch location sharing between family members in six weeks.

Step 1: declare failure. "It is six months after launch and the feature has been pulled. Why?"

Step 2: independent lists, then cluster. The team's lists cluster into five groups; the ratings are judgments made up for this example (invented):

Cluster	Example reasons	Likelihood	Impact
Misuse	Sharing used to monitor someone who did not freely agree	Medium	Severe
Teen workarounds	Teens turn off permissions, trust drops	High	Medium
Privacy backlash	A press story about location data handling	Medium	High
Reliability	Battery drain and stale locations	High	Medium
False reassurance	Parents assume the map is always current	Medium	High

Step 3: mitigations, signals and owners for the top clusters.

- Misuse: explicit opt-in from each person, a persistent visible sharing indicator, one-tap pause, and safety resources linked from the settings screen. Owner: product and trust and safety. Signal: reports of sharing without consent.
- Privacy: minimize retention, publish what is stored and for how long, and review with privacy counsel before launch. Owner: product and legal.
- False reassurance: show "last updated" time on every location. Owner: design.

Step 4: change the plan. The pre-mortem changed three launch criteria: opt-in from both sides, a visible indicator, and a kill criterion ("pause the launch if verified consent-violation reports exceed a set threshold in the first four weeks"). It also moved launch from all users to a [[staged-rollout]].

What this shows. The pre-mortem made a vague worry specific enough to act on, and it turned a design discussion into launch criteria. The design depth for this kind of feature is in the responsible-design check of C07.

6. Baseline, Strong, Standout

Prompt: "A new feature can ship in two weeks with known rough edges, or in six weeks fully polished. What do you do?"

Baseline answer

"I'd probably ship it in two weeks, because speed matters and we can iterate. Users prefer getting features early, and we can fix the rough edges later. I'd tell the team to prioritize the most important bugs. If it's really bad I'd wait."

What is wrong: no goal, no definition of "rough edges," no cost named, no reversibility, no evidence plan. "If it's really bad I'd wait" defers the whole decision.

Strong answer

"First I'd want to know the goal: is this a bet we are testing, or a commitment to customers? And what are the rough edges: cosmetic, or something that affects data or payments? Assuming the edges are minor and we can release behind a flag, I'd compare three options: ship in two weeks to everyone, ship in two weeks to 10% with a fix plan, or wait six weeks. Shipping to everyone gains learning speed but gives up polish for all users, and the cost lands on first impressions. Waiting gives up two to four weeks of learning. The staged option gains most of the learning while limiting the damage. I'd recommend the staged release, with a rule that anything touching data integrity blocks it. I'd measure task completion and support contacts for the 10%, and ship to everyone if they stay within a threshold we agree beforehand."

Standout answer

Everything in the Strong answer, plus:

"Before choosing, I'd separate two things the question bundles together: how bad the rough edges are, and who sees them. If a six-week schedule is slow because of a review queue, I would try to fix the queue, since that makes the tradeoff go away. If it is real, my deciding criterion is reversibility: a flagged release is easy to roll back, so I should decide quickly, whereas a polished release delays learning on the question that matters most, which is whether people use the feature at all. The cost of delay is four weeks of that learning. I'd put the downside in writing: the rough edges give up first-session trust for 10% of users, which I'd offset by choosing the segment least likely to be harmed, such as opted-in beta users. I'd set the exit criteria first: a rollback trigger on error rate and a hold on expansion if completion falls below the control. And I'd say what would change my recommendation: if the feature is the first thing new customers see, or if the edges affect accessibility, I'd wait."

What moves an answer up a band

From Baseline to Strong	From Strong to Standout
States the goal and defines the rough edges	Separates the bundled variables (severity, audience, cause of the delay)
Offers more than two options, including a staged one	Tests whether the tradeoff is real before accepting it
Names the cost in a sacrifice sentence	Names who bears the cost and offsets it deliberately

From Baseline to Strong	From Strong to Standout
Chooses a rollout mode and a decision rule	Sets exit criteria before launch and says what would reverse the recommendation

These map directly to signals 1 to 5 in section 1.

7. Company lens

All claims below are about what companies say publicly or what candidates commonly report; none is insider knowledge. Interview formats change [as of 2026-10].

Company lens: Google [as of 2026-10]. Google publicly describes hiring on general cognitive ability, role-related knowledge, leadership and "Googleness" Public [V: summaries citing Google's how-we-hire page; Google re:Work structured-interviewing guide]. Reported APM loops have rounds on product sense, execution and estimation, where a candidate who chooses and defends is distinguished from one who lists options Inferred. Practical consequence: end tradeoff answers with a recommendation and the sentence that names the cost.

Company lens: Amazon [as of 2026-10]. Amazon publishes its Leadership Principles, including Are Right, A Lot, Bias for Action and Dive Deep Public. Its shareholder letters in the mid-2010s describe one-way and two-way door decisions Public [V: Bezos 2015 shareholder letter (published April 2016), repeated in the 2016 letter, as quoted in AWS and secondary material], and its principle Have Backbone; Disagree and Commit covers speaking up and then committing Public. Using reversibility language is likely to be well received in an Amazon-style loop Inferred, as long as it is backed by a concrete classification.

Company lens: Meta [as of 2026-10]. Candidates commonly report product sense, execution and leadership rounds, with probing on metrics and tradeoffs Inferred [unverified: candidate-reported]. Expect follow-ups of the form "what would you measure to know it worked, and what could that metric hide?", which is where the second-order and counter-metric habits pay off.

Company lens: Microsoft [as of 2026-10]. Microsoft publicly emphasizes a growth-mindset culture Public. Reports suggest interviewers value candidates who say what they would learn from being wrong Inferred. Include the "what would change my mind" sentence and, in behavioral answers, what you changed afterward.

Company lens: Apple [as of 2026-10]. Apple publishes platform design guidelines and public statements on privacy Public [V: Apple Human Interface Guidelines (developer.apple.com) and privacy pages]. Third-party prep guides report team-specific loops, an emphasis on craft and ecosystem fit, and questions grounded in Apple products Inferred [unverified: candidate-reported]. In tradeoff answers, include polish and consistency as costs, not only speed and scope.

Company lens: Netflix [as of 2026-10]. Netflix's published culture material emphasizes context over control and individual ownership of decisions Public [V: Netflix culture memo, June 2024]. The case study CS-C02-02 is a reminder that even strong-culture companies make high-visibility decision errors; interviewers at any company like to hear what you would do differently.

Company lens: Stripe [as of 2026-10]. Stripe is widely described as having a strong writing culture Inferred [V: The Pragmatic Engineer, Inside Stripe's Engineering Culture Part 2 (strong culture of writing); Slab interview with Stripe's former Documentation Manager; any claim about rigor or precise thinking is inference]. Practical consequence: practice stating a problem, options and a recommendation in a few precise paragraphs, as in the decision memo (FW-C02-06), since clear reasoning on the page is a skill the company is described as valuing Inferred.

8. Common mistakes

#	Mistake	How to detect it	Fix
1	Answering with a solution before naming the problem	Your first sentence contains a feature	State the user, the situation and the outcome first
2	Options with no cost	You cannot finish "we give up..." for any option	Write the sacrifice before the benefit
3	Treating every decision as high stakes	You run a long analysis on a reversible choice	Classify reversibility first (FW-C02-04)
4	Treating every decision as cheap	You say "just try it" about a decision that affects customer money or data	Ask who cannot easily undo it
5	Confidence without a basis	You say "I'm sure" or give a precise number with no source	Give a label with a reason, and say what would raise it
6	Letting sunk cost steer	Your argument includes "we've already spent..."	Compare only future cost and future value (see C02.5)
7	Writing the problem to fit the solution	The problem statement uses the solution's nouns	Remove the solution words and re-test

#	Mistake	How to detect it	Fix
8	Stopping at the first-order effect	You never ask "and then what?"	Run three rounds of "and then what?" by stakeholder
9	False either-or	You accept "fast or good" without asking what makes fast worse	Name the specific cause and test whether it can be fixed separately
10	No stated reversal condition	You cannot say what would change your mind	Add "I would change my mind if ____" to every recommendation

C02.8 Anti-frameworks: patterns interviewers discount

Named frameworks are fine as scaffolding. These patterns are discounted because they replace reasoning with form `Inferred`. Each is an instance of `[[framework-theater]]` (C03).

Pattern	Why it fails	Do instead
Reciting an acronym method step by step regardless of the question	The structure is visible and the thinking is not	Use the steps that the question needs and say why you skip others
Scoring options 1 to 10 with no basis	False precision hides the real sacrifice	Use ordered comparisons and say what drives the order
The universal two-by-two	The axes were chosen after the answer	Choose axes from the goal, then place options
"It depends" and stop	Refuses the decision	Say what it depends on and then choose given your assumptions
"I'd ask the users" as the whole answer	Research is a step, not a decision	Name what you would ask, and what you would do with each answer
"Just A/B test it"	Not every decision can be tested, and some tests are not worth running	Say what the test would decide, how long it would take, and what you do meanwhile
"Build an MVP" with no riskiest assumption	The label stands in for a plan	Name the <code>[[mvp]]</code> assumption it tests (C15)
Metric lists	Ten metrics are no metric	Pick one primary <code>[[metric]]</code> , one <code>[[guardrail-metric]]</code> and say why (C05)

9. Mental models applied

Four models, each tied to a product decision. Two others that this chapter's decisions touch, `[[goodharts-law]]` and `[[jobs-to-be-done]]`, are owned by C05 and C06 and linked rather than redefined.

C02.5 Four models in use

Inversion (`MM-inversion`) is solving a problem by asking what would guarantee failure and then avoiding it. `[[inversion]]` works because failure modes are often easier to list than success factors. *Application*: a team designing a notification system asks "how would we

guarantee users mute every notification?" and lists: send during sleep hours, send low-value content, send too often, make settings hard to find. The design rules are the inverse of that list. Use inversion in a pre-mortem (FW-C02-05) and in the opening of any design answer.

Second-order effect (MM-second-order) is a consequence of a consequence, usually through other people's responses. [[second-order-effect]] thinking asks "and then what?" for each stakeholder. *Application:* a referral bonus raises signups (first order). Bonus-seekers sign up, never become active, and push support costs up (second order). The program's unit economics decline and the team tightens the bonus, which annoys genuine referrers (third order). A bonus tied to activation avoids most of the chain (CS-C02-01).

Local optimum (MM-local-vs-global) is the best result reachable by small changes from where you are, which may be far from the best result reachable by a different approach. [[local-optimum]] traps teams that tune what they have. *Application:* a signup flow improves by 2% a quarter through copy and button changes, then stalls. A step back to the [[problem-statement]] shows that most visitors do not understand the product, so the next step is a different first screen rather than another copy test. Detect it with plateauing gains. Respond by reserving some effort for a different shape of solution, then evaluating it with an [[experiment]] when you can.

Sunk cost (MM-sunk-cost) is cost already spent and not recoverable, which should not drive a forward-looking decision. [[sunk-cost]] bias makes teams finish projects that no longer deserve finishing. *Application:* a project is 70% built and new evidence shows weak demand. The question is whether the remaining 30% of the cost buys more value than the next best use of that effort, not how much has been spent. Two cautions: option value and learning are forward-looking and can legitimately justify finishing, and a team's morale is a real cost to weigh. The model removes the past from the calculation; it does not require you to ignore the future cost of stopping.

Engineering, design and data-science perspectives

These change the decision, so they belong here.

- **Engineering.** Reversibility looks different from the engineering side: a feature flag makes a change reversible for the code, but a data migration or a public API contract is not. Ask engineers "what is the cost of undoing this, and for whom?" before you classify.
- **Design.** User habits are a form of state. A design change can be technically reversible and still costly to reverse, because users have relearned an interface. Ask design what the relearning cost is.
- **Data science.** A decision's evidence is only as good as the question it answers. Ask what the data can and cannot tell you (observational data cannot separate cause from correlation, a point developed in C14) and what a plausible wrong answer would look like.

Checklist: decision hygiene

1. Did I state the decision and the goal?
2. Did I separate the problem from the proposed solution?
3. Did I include a smaller option and "do nothing"?
4. Did I write each option's cost, and who bears it?
5. Did I classify reversibility and the cost of delay?
6. Did I state my confidence with a reason?
7. Did I say what would change my mind?
8. Did I name the first step and the earliest signal that I am wrong?

10. Case studies

CS-C02-01 · The referral bonus · Illustrative · Beginner

Fictional composite. Every number is invented.

Situation. A budgeting app wants 50,000 new signups this quarter (invented). The growth lead proposes doubling the referral bonus from \$5 to \$10 per signup (invented), paid when the invited friend registers.

Constraints. The quarter ends in 10 weeks. Marketing budget allows an extra \$150,000 (invented). The company's rule is that a new customer should pay back acquisition cost within 12 months.

Data. Last quarter, referral signups were 8,000 (invented) and 35% of them were still active after 30 days (invented). Paid-channel signups had 55% 30-day activity (invented). Support contacts per referred user were 0.4 against 0.2 for paid (invented).

Problem definition. The business needs more active users, not more registrations. The proposal optimizes registrations.

Analysis. First-order effect: doubling the bonus should raise referral signups. Second-order effects, asked stakeholder by stakeholder: bonus-seekers will register accounts they will not use, which lowers activation and raises support load; existing genuine referrers will notice the program looks transactional; and the finance rule on payback will be strained because cost per active user rises. The last effect shows up only if cost is measured per active user rather than per signup, which is the [[goodharts-law]] trap (C05).

Options.

Option	Optimizes	Costs
A. Double the bonus on registration	Signup count	Lower activation, higher support, payback strain
B. Keep \$5 on registration, add \$10 when the friend completes a first budget	Active users	Slower payout; fewer registrations

Option	Optimizes	Costs
C. Do nothing	No spend	Misses the target

Tradeoffs. B gives up raw signup volume to protect the metric the business cares about. It is reversible: the bonus structure can be changed after two weeks with a staged rollout to half of users.

Decision. Choose B, test it on 50% of referrers for three weeks, and track cost per active user and support contacts per referred user as guardrails.

Outcome. In this fictional scenario, B produced 12% fewer referral signups than A would have (invented) but 30% more users active at 30 days per dollar (invented). The company missed the raw signup target and met the activation goal that it later adopted as the quarterly metric.

Alternative outcomes. If activation among bonus-seekers had turned out high, A would have been right, and the test would have shown it. If the finance rule had been looser, the team might have run A and B side by side. The point of the case is the method: first-order thinking would have picked A and second-order thinking changed the decision.

CS-C02-02 · Netflix, 2011: a price change and a split that were reversed · Reconstructed · Advanced

Facts below come from Netflix's shareholder letter for the third quarter of 2011 (filed with the US Securities and Exchange Commission) and Associated Press reports of 10 and 25 October 2011, which quote Netflix's chief executive and report the company's September guidance; the analysis is inference. No internal Netflix information is used. Re-read in October 2026.

Situation. In July 2011 Netflix moved to charge separately for streaming and for DVDs by mail. Reporting describes the effective price of the combined plan rising from about \$10 to about \$16 a month, a rise of as much as 60% for some customers. In September 2011 the company announced that its DVD service would become a separate brand, Qwikster, with its own website and account. On October 10, 2011 it reversed the Qwikster plan and said customers would keep one website and one account, while keeping the July prices.

Constraints. Netflix's shareholder letter says the company had failed to explain the rising cost of expanding streaming content alongside steady DVD costs, so that many customers saw it as greedy, and its chief executive described DVD-by-mail as a business in steady decline and streaming as the future. The business had millions of subscribers on a combined plan.

Data. Netflix reported about 24.6 million US subscribers at the end of June 2011 and about 23.8 million at the end of September, a drop of about 800,000, against a loss of about 600,000 that the company had forecast in September. The share price fell about 35% the day after the October results and by more than half between July and October, according to Associated Press reports.

Problem definition. *Inferred.* Two separate problems were bundled: the economics of content costs, which called for a price change, and the longer-term shift from DVD to streaming, which called for operational separation. The customer problem was different from both: customers valued one place to get films and one account.

Analysis. *Inference.* Three decision-quality questions apply.

1. *Reversibility.* A price increase is announced publicly and customers who leave are not guaranteed to return. The Qwikster plan was in effect a one-way announcement: reversing it was possible, but it cost credibility. The company did reverse Qwikster within weeks and kept the price change, which suggests it judged the price decision to be the necessary one.
 2. *Second-order effects.* The first-order effect of the split was operational clarity. The second-order effect was that customers who liked a single account now faced two, and that the split signaled the company was moving away from the combined service they had chosen.
 3. *Staging.* The decisions appear to have been announced to all customers rather than tested with a subset, which would have measured customer reaction before commitment.
- Inferred.*

Options. *Reconstructed.*

Option	Optimizes	Costs
A. Price change and split, as done	Economics and strategy clarity	Customer trust and subscribers, as reported
B. Price change only, communicate the content-cost reason	Economics	Some cancellations; no brand split
C. Price change, then test a separate DVD brand with a subset	Learning before commitment	Slower strategic separation

Tradeoffs. Speed and strategic clarity were traded for customer convenience and trust, in the open, with little ability to undo the first impression.

Decision. *As reported.* Reverse the Qwikster separation; keep the July price.

Outcome. *Documented through October 2011.* Subscriber losses and a large share-price fall, as above. Later years are outside this case.

Alternative outcomes. *Inference.* Option C might have shown the account-friction problem at small cost. A pre-mortem (FW-C02-05) would likely have listed "customers do not want two accounts" as a high-likelihood failure, because it is exactly the sort of reason a team imagines when told the plan failed. We cannot know whether it would have changed the decision.

Lesson for an interview answer. When you recommend a change that customers will see, say how reversible it is for customers, not only for engineers, and say what small test would tell you before you commit.

11. Interview questions and model answers

Six questions, two per difficulty. All model answers are *Illustrative*; numbers are invented. Each was written against the five signals in section 1.

IQ-C02-01 · Beginner · product thinking · PS

Prompt. Users keep asking for a dark mode in your app. Would you build it?

Rubric.

- **Baseline:** "Yes, users are asking for it, and many apps have it." No segment, no cost, no measure.
- **Strong:** Asks who is asking and why, estimates demand from available data, names the cost, compares with alternatives and decides with a criterion.
- **Standout:** Separates a preference from a need, finds the smallest reversible step, defines a metric and a counter-metric, and says what would change the answer.

Model answer. I would start by restating the need behind the request. People who ask for dark mode are usually doing one of three things: reading at night and wanting less glare, saving battery on some displays, or needing higher contrast for accessibility. Those are different problems, and the third is a need rather than a preference.

Next I would check the evidence we already have. How many of the requests mention night use, and from which segment? What share of sessions happen between 9pm and 6am? Request counts also overstate demand, because the people who ask are the people who care most, so I would pair them with usage data rather than rely on the count alone.

Options of different kinds: follow the phone's system theme setting, which is the smallest option and covers a large share of the need; build an in-app toggle with its own theme; do nothing and invest the effort elsewhere. The system-theme option costs the least and is easy to reverse. A full custom theme costs more design and testing time, and it takes engineering effort away from the work that is competing for it.

My recommendation is to follow the system setting first, because it serves most of the need at a fraction of the cost, and to expand to a toggle only if users with the system setting turned on still ask for more control. I would measure how many users have the new theme active and whether night-time session length changes for them, with crash rate and support contacts as guardrails. I would change my mind if the requests cluster in an accessibility segment: contrast then becomes a quality requirement rather than a nice-to-have.

Follow-ups.

- **Q.** What if most of the requests come from users who already have their phone set to dark? **A.** "Then the request is probably about control, not about the theme, because those users already get a dark appearance in other apps. That makes an in-app toggle more plausible. I would check what share of users override the system setting in other apps and ask five of the requesters what they want to control. Once a theme system exists, a toggle adds little cost, so the decision becomes cheap; I would still ship it behind the same guardrails."

- **Q.** How would you measure success when dark mode barely moves revenue? **A.** "I would not expect revenue to move, so I would measure the jobs it serves: adoption among night-time users, night-time session length or task completion for them, and support contacts or reviews that mention glare or contrast. I would test retention against a holdout and treat any gain as a bonus, not a promise. If only satisfaction and accessibility complaints improve, that is a legitimate result, and I would describe it as a quality investment and not a growth lever."

IQ-C02-02 · Beginner · prioritization · EX

Prompt. Sales asks you to build a custom report for one large customer who is threatening to leave. Do you build it?

Rubric.

- **Baseline:** "Yes, the customer is important" or "No, we never build one-offs," with no reasoning.
- **Strong:** Sizes the customer's value, checks whether the need is shared, scopes the smallest version, names the cost to the roadmap, and decides by a stated criterion.
- **Standout:** Separates the request from the underlying job, tests whether a configurable version serves others, time-boxes the work, names the precedent cost, and says what would change the answer.

Model answer. I would not answer yes or no yet. I would ask what the customer is trying to do with the report, because "a custom report" is one way to meet a need, and the need may be shared by other customers.

Then I would size three things. First, the customer's value: say the account is 6% of annual revenue and renews in three months (invented). Second, how many other customers have asked for something similar, from support and sales notes. Third, the cost: two engineers for three weeks (invented), which is work that would otherwise go to the roadmap.

My options differ in kind. Build exactly what was asked, gaining the account and giving up three weeks of roadmap plus a precedent that large customers get one-offs. Build a configurable export or report builder that covers this case and others, which costs more now and serves more customers. Offer an interim workaround, such as a data export and a template the customer's analyst can use this week, which is cheap and reversible.

I recommend the workaround this week, to remove the immediate threat, and a time-boxed scoping of the configurable version, with the decision based on how many other customers show the same need. The cost is that the customer gets less than they asked for at first, and I would explain why and when they get more. I would change the recommendation if the contract names the report, if the account is large enough that losing it changes the company's plans, or if scoping shows the configurable version costs about the same as the one-off.

Follow-ups.

- **Q.** Sales says the customer will leave on Friday unless you commit. What do you say? **A.** "I would ask sales what the customer actually said and what would satisfy them this

week. A dated commitment to a defined interim (an export and a template) plus an executive call often does. I would not promise the full report under a deadline I cannot verify; I would state what I can deliver and by when, in writing. If the customer truly requires the full report, I would take it to my manager with the revenue at stake and the cost, because it is then a leadership tradeoff about the roadmap, not mine alone."

- **Q.** How do you stop this from becoming a precedent where every large customer gets a one-off? **A.** "With a written rule applied consistently: custom work is considered when the need is shared by several accounts or the account passes a revenue threshold (say 5% of revenue, invented), it is time-boxed, and it is built as a configurable capability where possible. I would count such requests and their cost each quarter. Sales also gets a clear path, so they can say 'not as a one-off, but here is how it can be considered' instead of promising on the spot."

IQ-C02-03 · Intermediate · execution · EX

Prompt. Engineering proposes removing a feature that is rarely used but that a few large enterprise customers rely on. How do you decide?

Rubric.

- **Baseline:** Counts users and removes it because "usage is low," or keeps it because "customers might leave."
- **Strong:** Measures revenue and strategic weight of the affected customers, estimates the carrying cost, considers alternatives, and plans the removal.
- **Standout:** Classifies the decision as one-way for the customers, reduces the irreversibility with a notice period and a migration path, and states the conditions that would make keeping it right.

Model answer. Usage counts are the wrong first measure: a feature used by 2% of accounts can be used by 20% of revenue. So I would ask four questions. Which customers use it, and what share of revenue and renewals do they represent (invented example: 5 accounts, 9% of annual revenue)? What does it cost to keep, in engineering time, incident risk and the drag it adds to other changes? What do the customers do with it, and is there an alternative path? And what is the decision's reversibility?

For the customers, removal is a one-way door. They have built workflows on it. That points to the mode "decide with evidence and reduce the irreversibility," not "decide quickly."

My options: keep it as is; keep it but freeze it, with no new investment; replace it with a supported alternative and migrate customers; remove it with a long notice period. Keeping it as is gives up engineering capacity. Removal gives up the revenue risk and the trust of the customers, whose renewals may be tied to the feature. Replacing it asks us to build something, and costs the most effort but is the only option that serves both sides.

I would recommend freezing it now, investigating a replacement path with the five customers, and deciding on removal when we know the replacement cost and each customer's migration effort. If the carrying cost turns out to be large and the replacement

path is cheap, I would remove it with at least a quarter's notice and a named migration contact. I would change my mind if the customers' contracts name the feature or if the replacement cost exceeds the revenue at risk.

Follow-ups.

- **Q.** The five customers say they will never migrate. Do you still remove the feature? **A.** "Never" under pressure often means 'not for free', so I would ask what migration would take and what we could offer: tooling, a named migration contact, a longer notice, perhaps a price adjustment. Then I would compare the carrying cost with the revenue at risk (9% of annual revenue in the example, invented) after expected churn, with finance. If the revenue at risk is larger than the savings, I keep it frozen. If it blocks other work and the replacement path is cheap, I remove it with a long notice. Either way it is a business decision I would bring to leadership with the numbers."
- **Q.** Engineering says the feature causes most of the team's incidents. How does that change your decision? **A.** "It turns the carrying cost from abstract into measurable: incidents per month, engineer-hours on call, and the delay to other work. If, say, 40% of on-call load comes from this feature (invented), freezing is not enough. I would choose between hardening it, which is cheaper and keeps it, and building the replacement path, using the same customer conversations. Doing nothing has a cost too, and I would put that number next to the revenue at risk."

IQ-C02-04 · Intermediate · product thinking · PR

Prompt. Customer support costs have doubled in a year, while the customer base grew 30%. What do you do?

Rubric.

- **Baseline:** "Add a chatbot" or "hire more efficient agents," with no diagnosis.
- **Strong:** Decomposes cost into volume and cost per contact, finds the drivers, and addresses the largest.
- **Standout:** Separates fixed facts from conventions, computes the per-customer change, finds root causes upstream of support, and chooses between deflecting contacts and removing the reason for them.

Model answer. I would start with arithmetic, because "doubled" and "grew 30%" are not the same thing. If total cost doubled and customers grew 1.3 times, cost per customer rose by about 54% (2 divided by 1.3, which is 1.54). So growth explains part of the increase, but not most of it.

Decompose cost. Cost equals contacts times cost per contact. Contacts equal customers times contact rate. Cost per contact equals handling time times labor cost per minute plus tooling. I would find which factor moved: did the contact rate rise, did handling time lengthen, or did the labor cost change? Suppose the data (invented) show the contact rate per customer up 40%, handling time up 10%, and the rest from labor.

The contact rate is the largest driver, so I would examine its reasons. Break contacts by reason and by customer cohort: billing questions, delivery issues, product defects, "how do I" questions. Say billing questions are 35% of the increase (invented). The first-principles question is: what is the user trying to do, and why can they not do it without help? If the billing page is confusing, the fix is a clearer page, which removes contacts rather than handling them.

The options differ in kind. A chatbot deflects contacts at the support layer, is quick to build, and leaves the root cause. Improving the billing page removes contacts at the source, takes longer, and benefits customers who never contacted support. Training agents reduces handling time and affects the smaller factor. I would recommend working on the top two contact reasons at the source, with a limited deflection tool for the long tail. I would measure contacts per customer by reason, not total cost, with customer satisfaction as the [[guardrail-metric]] so that cost cuts do not come from worse service. I would change course if the top reasons turn out to be outside our control, such as carrier delays.

Follow-ups.

- **Q.** Billing questions are the largest driver of the extra contacts. What do you do first? **A.** "I would split billing contacts by cause: invoice confusion, failed payments, refunds, plan changes. I would fix the most frequent cause at the source, for example a clearer invoice and a self-serve refund path, and measure contacts per 1,000 customers by reason before and after. I would also check whether contacts are first contacts or repeats. Failed payments are worth a look because they create both contacts and cancellations, so better retries and notifications may cut both at once."
- **Q.** Leadership wants a chatbot now. Do you build it? **A.** "Not as the first step. A bot deflects some contacts and leaves the cause, and a bad one raises repeat contacts and frustration. I would fix the top two root causes first, then use a bot or help content only for questions that stay high-volume and answerable. I would measure deflection honestly: resolved with no repeat contact within seven days, plus satisfaction. If the resolution rate is under about 40% (invented) or repeat contacts rise, I would stop and say why."

IQ-C02-05 · Hard · product thinking · PS

Prompt. Your VP asks the team to grow daily active users by 20% this quarter. How do you respond?

Rubric.

- **Baseline:** "Run growth campaigns and add notifications," with no discussion of the target.
- **Strong:** Asks what the target is for, decomposes the metric, proposes options and guardrails.
- **Standout:** Anticipates how the metric will be gamed, separates real value from activity, proposes counter-metrics and staged tests, and negotiates the target with a reason.

Model answer. I would first ask what the target is meant to achieve, because daily active users is a proxy. If the underlying goal is more people getting value, then there are targets that measure it more directly, and I would raise that with the VP rather than silently optimize a proxy. If the VP has a reason for this one, such as a board commitment, I would take it as given and work within it.

Decompose the metric. Daily active users equal new users plus returning users, minus those who stop. I would look at what drives each, and where the cheapest honest gain lies: activation of new users, resurrection of lapsed ones, or frequency among those who already use the product.

Then I would think through second-order effects, because this target invites gaming. Notifications can raise daily activity by prompting opens that deliver no value, which increases the count and erodes trust and opt-in rates later. That is a classic [[goodharts-law]] outcome. So I would pair the target with counter-metrics: notification opt-out rate, sessions with a meaningful action, and 30-day retention.

My plan would test two or three levers in order of honesty about value: a better first-week experience, a reminder tied to something the user said they care about, and a content improvement. Each is tested as an [[experiment]] with the counter-metrics watched. I would tell the VP up front what I expect: a gain of a certain size from the honest levers (a range, with low confidence until we have a first read), and what extra gain would come only from tactics I would not recommend. If the full 20% is reachable only by tactics that damage retention, I would propose a smaller target for DAU with a retention floor. I would change my plan if the first experiments show frequency, not activation, is where the headroom is.

Follow-ups.

- **Q.** What if the VP says the number is fixed: 20% and nothing else? **A.** "I would accept the number and protect what it stands for. I would propose the levers (activation, resurrection, frequency), the counter-metrics and a forecast range with my confidence, and ask that the counter-metrics be reported alongside the target. I would also say which tactics I will not use, such as notifications that deliver no value. If it is a board commitment, I would ask what the board cares about, since a related measure may be acceptable, but I would not reopen the question repeatedly."
- **Q.** Which lever would you try first, and how would you know the growth is honest? **A.** "The first-week experience, because new-user activation compounds and is least likely to be gaming. Honest growth shows in cohorts: new users activating and returning at the baseline rate or better, sessions with a meaningful action rising with total sessions, opt-outs and complaints flat, and 30-day retention of the added users similar to older cohorts. If the extra daily users have short sessions and high opt-outs, the number is inflated."

IQ-C02-06 · Hard · behavioral · BE

Prompt. Tell me about a decision you made with incomplete information.

Rubric.

- **Baseline:** A story with no stated options, no stated information gaps, and an outcome that is the whole point.

- **Strong:** Names the options, the missing information, the decision rule and the result, and says what was learned.
- **Standout:** Shows how the candidate sized the cost of being wrong, made the decision reversible, set a trigger to revisit it, and separates decision quality from the outcome.

Model answer. *Illustrative: a composite story, numbers invented.* "I was the PM for a scheduling feature at a software company. Two weeks before a planned release, a customer-facing team told me that a partner calendar integration might behave differently in time zones than we had tested. We could not reproduce it, and the partner's documentation was unclear. I had three options: delay the release by three weeks to get the partner's confirmation, release with the integration enabled for everyone, or release without the integration and add it later.

"I defined the cost of being wrong in each case. A wrong meeting time is more than an annoyance, because someone misses a meeting. That made the integration the risky part, and the rest of the release was fine. A delay would have cost three weeks on a feature that customers had asked for. Releasing the rest without the integration would give up about a third of the expected use but carried no risk of wrong times.

"I chose to release without the integration, put the integration in a small beta with 30 volunteer customers (invented), and set a rule: expand when we saw no time-zone errors across 500 scheduled meetings (invented), and pause if one was reported. I told my manager at the time that I was about 60% confident the integration was fine and that this was a judgment, not a measurement.

"The beta found one edge case with a half-hour time zone offset. We fixed it and expanded the integration five weeks later. I would make the same decision again. If the problem had not appeared, I would still have given up about five weeks of integration use, which I counted as a known cost. What I changed afterward was to ask partners for sandbox environments early in the project, because the information gap was avoidable."

Follow-ups.

- **Q.** What would you have done if the beta had found several time-zone errors? **A.** "The rule I set in advance was to pause on one reported error, so I would have paused the integration for everyone outside the beta, found the cause with the partner and told the beta customers what happened. The release without the integration was already out, so the exposure was limited to 30 customers. I would have repeated the beta after a fix under the same rule, and reconsidered whether the integration was worth shipping at all if errors persisted."
- **Q.** You told your manager you were about 60% confident. Why say that out loud? **A.** "Stating it turns a feeling into something my manager can challenge or accept. It shows I know the decision is a judgment, and it gives me a way to check my calibration later: across many decisions, were my 60s right about 60% of the time? It also changes how the team prepares: at 60% they want the beta and a rollback; at 95% they would just ship."

12. Exercises

Ten exercises. Each has a model solution and an alternative. Numbers in scenarios are invented.

EX-C02-01 · Reframe three requests

Problem. Rewrite each request as a need, a segment and an outcome: (1) "Add a search bar to the dashboard." (2) "Make onboarding shorter." (3) "We need a mobile app."

Context. You are a PM at a B2B analytics product. Requests came from sales, an executive and a customer.

Data. Support logs show 18% of tickets are "can't find report" (invented). Onboarding completion is 52% (invented). 38% of sessions are on mobile browsers (invented).

Constraints. You have 20 minutes and no new research budget.

Expected approach. For each, apply the product thinking chain: who, what is happening, why it matters, evidence, and what is not the problem.

Solution. (1) Need: users who cannot locate a saved report. Segment: occasional users with many reports. Outcome: fewer "can't find report" tickets and faster time-to-report. Not the problem: the search bar itself, since a naming or navigation change may do it. (2) Need: new users reach first value. Segment: users who drop before completing setup.

Outcome: raise completion from 52%. Not the problem: the number of steps; fewer steps may not help if one step is the barrier. (3) Need: users on phones cannot do something they need. Segment: the 38% on mobile browsers. Outcome: task completion on mobile. Not the problem: "an app"; a responsive web view might serve the need.

Alternative solution. Write each as a [[problem-statement]] using the five-row template and share it with the requester to check you captured their need.

Tradeoffs. Time spent reframing delays the responses to the requesters; a quick restatement back to them preserves the relationship while you check.

EX-C02-02 · First-principles cost decomposition

Problem. A food-delivery startup charges \$3.50 per delivery and loses money. Decompose the cost of one delivery from first principles and identify what is fixed and what is convention.

Context. Average delivery is 3 km; couriers are paid per delivery.

Data. Courier pay \$2.80 per delivery (invented), payment processing 2.5% of a \$20 order (invented), support \$0.30 per order (invented), insurance \$0.15 per order (invented), average courier waits 6 minutes at the restaurant (invented).

Constraints. You cannot change the restaurant's prep time.

Expected approach. List the cost components, label each as fixed floor or convention, then rebuild options.

Solution. Cost per order: courier \$2.80, processing \$0.50, support \$0.30, insurance \$0.15, which totals \$3.75, against \$3.50 charged. Fixed floors: some courier pay (distance and time are real), processing fees (set by networks), insurance. Conventions: paying per delivery regardless of batching, charging a flat fee regardless of distance, a six-minute wait at the restaurant. Options: batch two deliveries per trip, which cuts courier pay per order; distance-based fees; encourage restaurants to signal ready times. Batching changes the largest component and is the first one to test.

Alternative solution. Raise the fee to \$4.00. It fixes the arithmetic and gives up conversion; test it as an [[experiment]] before committing.

Tradeoffs. Batching reduces cost and lengthens delivery times for some customers.

EX-C02-03 · The tradeoff table

Problem. Build a tradeoff table for this decision: a photo-sharing app can add video uploads in six weeks, add better photo editing in four weeks, or improve upload reliability in three weeks.

Context. The goal this quarter is retention of existing users. Team of five engineers.

Data. Uploads fail for 6% of attempts (invented). 22% of active users edit photos weekly (invented). Competitor apps support video.

Constraints. Only one of the three is possible this quarter.

Expected approach. Fill the table (optimizes, costs, reversibility, evidence needed) and write the recommendation line.

Solution. Upload reliability: optimizes retention among frustrated users, costs the chance to match a competitor, fully reversible, evidence available now (failure rate by cohort). Photo editing: optimizes engagement for a quarter of weekly users, costs reliability work, reversible. Video: optimizes competitive parity, costs the longest build and the highest risk, hard to reverse because storage and moderation costs commit the company. Recommendation: "Fix upload reliability, which protects retention by giving up the video parity move this quarter; reversible; I would switch to editing if failure rates are below 2% after the fix."

Alternative solution. Start video as a limited beta in parallel using one engineer, accepting a slower reliability fix.

Tradeoffs. Parity pressure is real, so the recommendation must say what would pull video forward.

EX-C02-04 · Classify five decisions

Problem. Classify each decision using FW-C02-04: (a) change an email subject line; (b) raise subscription price by 10% for existing users; (c) migrate user data to a new database; (d) rename a menu item; (e) shut down a free tier.

Context. You are the PM of a subscription product.

Data. None beyond the descriptions.

Constraints. Write one sentence of reasoning each.

Expected approach. For each, ask: reversible, error costly, waiting costly; then pick a mode.

Solution. (a) Reversible, low error cost: decide now. (b) Hard to reverse once users have seen it; high error cost; waiting cost depends on margin: stage by cohort and communicate. (c) Reversible only with a backout plan; high error cost: decide with evidence and a rehearsed rollback. (d) Reversible: decide now. (e) Hard to reverse, high error cost: decide with evidence, give notice, offer a path.

Alternative solution. Add the audience dimension: reversibility for users, for engineering and for the brand can differ.

Tradeoffs. Over-classifying small decisions wastes time; the exercise is to be quick on (a) and (d).

EX-C02-05 · Inversion drill

Problem. Use inversion to design the first-run experience of a language-learning app. List ten ways to guarantee users never complete a first lesson, then invert them.

Context. Users arrive from an ad; many are casual.

Data. 60% of installs never open a lesson (invented).

Constraints. Ten minutes.

Expected approach. List failure causes, then derive design rules.

Solution. Failures: require account creation first; ask for notification permission immediately; show a long placement test; hide the first lesson; use jargon; require a payment method; load slowly; show an unclear goal; ask personal questions; show an error on first tap. Inverted rules: let users try a lesson before signing up; ask for permissions after value; use a short placement; make the first lesson one tap; plain language; no payment until after value; fast loading; clear goal; ask only what you need; test first-tap paths.

Alternative solution. Run the same exercise with the team as a pre-mortem before the next release.

Tradeoffs. Skipping account creation reduces data capture; the cost is that you learn less about trial users until later.

EX-C02-06 · Second-order effects of a feature

Problem. A ride-hailing app adds a "tip your driver" prompt at ride end. Map first-, second- and third-order effects for riders, drivers and the company.

Context. The app currently has no in-app tipping.

Data. None.

Constraints. Include at least one effect you would measure with a counter-metric.

Expected approach. Ask "and then what?" three times per stakeholder.

Solution. Riders: first, some tip; second, others feel social pressure and rate drivers lower or avoid the prompt; third, rider satisfaction and rating accuracy shift. Drivers: first, extra income; second, drivers favor rides likely to tip; third, service inequality across neighborhoods. Company: first, driver satisfaction rises; second, pressure to cut fares or fees may follow; third, regulators may ask about earnings. Counter-metrics: ride-cancellation rate by area, rider rating distribution, tip rate by area.

Alternative solution. Test the prompt in two cities with different norms before a wider launch.

Tradeoffs. The prompt helps drivers and risks rider goodwill; the choice is where to place it (end of ride versus the receipt).

EX-C02-07 · Escaping a local optimum

Problem. An online store's checkout conversion has improved 1% a quarter for a year and now plateaus at 62% (invented). Diagnose the plateau and propose a step outside the current approach.

Context. Tests so far: button copy, field order, trust badges.

Data. 55% of cart abandoners say "just browsing" in exit surveys (invented).

Constraints. Engineering capacity is limited to one large project.

Expected approach. Ask what the incremental tests assume and what a different shape of solution would change.

Solution. The tests assume the problem lives inside checkout. Exit surveys suggest many abandoners were never ready to buy, so the lever is upstream: saved carts, price-drop reminders, or comparison tools. A larger project is a "save for later" flow that returns people when ready. Test it as an [[experiment]] on carts above a certain value.

Alternative solution. Keep optimizing checkout for the 45% who say they intended to buy.

Tradeoffs. A new flow takes longer and may not move the headline conversion rate, since its users were counted as abandoners before.

EX-C02-08 · Sunk cost

Problem. A project to build an in-app calendar is 70% complete after four months. New research shows most users prefer to sync with their existing calendar. What do you do?

Context. Four engineers have worked on it; three more months remain.

Data. Interviews with 15 users (invented): 12 prefer syncing, 3 want a built-in calendar.

Constraints. The team will ask you to finish "since we are almost done."

Expected approach. Ignore costs spent. Compare the remaining cost and value of finishing with the best alternative, including partial reuse.

Solution. The remaining three months should compete with a sync integration. Check what part of the calendar work is reusable for sync (date logic, event model). If the integration reuses 40% of it, finishing the calendar is not the default. Recommend pivoting the remaining effort to sync, and keep the three users who want a built-in calendar in a follow-up interview.

Alternative solution. Finish a minimal calendar if learning shows that it is a differentiator for a segment you value.

Tradeoffs. Stopping costs morale and the sense of finishing; set a clear message about what the team learned and what will ship.

EX-C02-09 · Write a one-page decision memo

Problem. Write a decision memo for: a news app must decide whether to add a paywall after five free articles a month, or keep an ad-only model.

Context. Ad revenue per user has fallen 15% in a year (invented). The CEO wants a recommendation in a week.

Data. 4% of monthly readers read more than five articles (invented). 1.2% of readers have ever subscribed to anything in the app category (invented). Ad revenue is 90% of income.

Constraints. One page; include reversibility and what would change the recommendation.

Expected approach. Use FW-C02-06.

Solution. Decision: whether to meter after five free articles. Recommendation: pilot a meter at eight articles for 10% of readers for six weeks, which gives up some ad impressions and tests willingness to pay with limited downside. Context: ad revenue is falling and the meter touches only 4% of readers. Options: full paywall, meter pilot, ad-only with more ad load, do nothing. Criteria: revenue, reader trust, reversibility. Reversibility: a meter can be loosened, but readers who leave may not return. Would change: if conversion in the pilot is below the level needed to offset lost ad revenue, shift to improving ad yield.

Alternative solution. Run a survey of readers on willingness to pay; weaker evidence because of the [[say-do-gap]].

Tradeoffs. A pilot is slower than a full launch and costs some ad revenue.

EX-C02-10 · Run a pre-mortem

Problem. Run a pre-mortem on launching a "buy now, pay later" option in a shopping app.

Context. Launch in eight weeks with a lending partner.

Data. None.

Constraints. Produce a ranked list, mitigations and at least one change to the launch plan.

Expected approach. Declare failure, list reasons independently, cluster, rank, assign mitigations and owners.

Solution. Clusters: customers over-borrow and default (high impact), partner declines too many applicants (medium), checkout friction raises abandonment (medium), regulatory disclosure gaps (high impact, [varies by market]), support cannot handle disputes (medium). Mitigations: clear limits and repayment reminders, partner approval-rate monitoring, a one-tap checkout test, legal review of disclosures, a dispute path with a named owner. Change to plan: launch to a small share of customers with a hold on expansion if default and complaint rates exceed set thresholds.

Alternative solution. Run the pre-mortem separately with the partner and engineering, then merge.

Tradeoffs. A cautious launch reduces learning speed and protects customers.

13. Practice protocol

Timed drill (twice a week for three weeks).

1. Pick a prompt: a tradeoff, a "should we build it" or a "why that decision" question (use IQ-C02-01 to IQ-C02-06 and the exercises above).
2. Set a timer for ten minutes. Speak aloud or write the answer in the Type A structure from section 4.
3. Record yourself. At minute ten, stop.
4. Score with the checklist below.

Feedback checklist (score each item yes, partly, no).

1. Did I state the decision and the goal before options?
2. Did I separate the problem from the solution?
3. Did I include a smaller option and "do nothing"?
4. Did I name each option's cost in a sacrifice sentence?
5. Did I say who bears the cost?
6. Did I classify reversibility and the cost of delay?
7. Did I state confidence with a reason?
8. Did I say what would change my mind?
9. Did I end with a recommendation in one sentence?
10. Did I stay within ten minutes?

With a partner. Ask your partner to interrupt with "why not the other option?" at minute five. Note whether your second response is reasoning or defence.

Answer journal. For each drill, record the date, the prompt, your score on the ten items, the one item you will fix next time, and the sentence you wish you had said. After six drills, look for the item you miss most often and drill that one.

14. Implementation playbook

Apply the chapter at your current job in the next two weeks. Each deliverable doubles as an interview story.

Days 1 to 3. List your last five meaningful decisions at work. Classify each using FW-C02-04, and note whether you gave it the right speed. Keep the two that show a clear tradeoff.

Week 1. Take one live feature request and run it through the product thinking chain (FW-C02-01). Write the problem using the five-row template. Share it with the requester and note whether they agree it captured the need.

Week 1. Write one decision memo (FW-C02-06) for a real decision that is about to be made, even if you are not the decision owner. Ask your manager for feedback on the "what would change this" line.

Week 2. Run a pre-mortem (FW-C02-05) on your next launch with the team. Bring a list of changes you propose, and an owner for each.

Week 2. Start a prediction log: before the next experiment readout or launch, write your prediction and confidence. Score it afterward.

Deliverables that become stories: the decision memo (a decision made with incomplete information), the pre-mortem (a risk you caught early), and the reframed request (customer or problem focus). Keep the numbers real; do not inflate them.

15. Key takeaways and further reading

Key takeaways

1. Start from the user's problem and the outcome you can measure, not from the proposed solution, unless the decision is cheap and reversible.
2. Put the sacrifice in the sentence: every option has a cost, and a choice without one is unfinished.
3. Classify reversibility, error cost and cost of delay before you decide how much evidence to demand.
4. Ask "and then what?" for each stakeholder, and ask what would guarantee failure, before you commit.
5. Close every recommendation with what would change your mind.

Further reading

- Donald G. Reinertsen, *The Principles of Product Development Flow* (2009). The economic framing of decisions, including cost of delay.
- Gary Klein, "Performing a Project Premortem," *Harvard Business Review*, September 2007. The source of the pre-mortem technique in FW-C02-05.

- Daniel Kahneman, *Thinking, Fast and Slow* (2011). A broad account of judgment biases, including overconfidence and the sunk-cost fallacy.
- Annie Duke, *Thinking in Bets* (2018). Separating decision quality from outcome, and stating confidence.
- Marty Cagan, *Inspired* (2nd edition, 2017). Product discovery as a problem-first discipline.
- Jeff Bezos, Amazon shareholder letters for 2015 (published April 2016) and 2016, which describe one-way (Type 1) and two-way (Type 2) door decisions; the letters are on Amazon's investor relations site [V: letters as quoted in AWS and secondary material; the letters themselves were not opened].